

1 Background

1.1 Dimensions

The constants in Table 1 define the sizes of the various arrays described later. These are from the `Config struct`.

Math	Code	Explanation
n_L	<code>n_layers</code>	number of layers in the network
n_S	<code>seq_len</code>	max sequence length
n_D	<code>dim</code>	transformer dimension
n_H	<code>n_heads</code>	number of query heads
n_{DKV}	<code>kv_dim</code>	$(n_D \times n_{HKV})/n_H$
XXXX	<code>hidden_dim</code>	for ffn layers
n_V	<code>vocab_size</code>	vocabulary size
n_{HKV}	<code>n_kv_heads</code>	number of key/value heads
n_{H_0}	<code>n_heads * head_size</code>	
n_{H_1}	<code>n_kv_heads * head_size</code>	

Table 1: Dimensions

1.2 Definitions

Definition 1 A scalar, M_0 is defined using a type t_0

Definition 2 A vector or 1D matrix, M_1 , is defined using a type t_1 and positive integer n_1 , representing the dimensions/shape.

- $M_1[j]$ is a scalar of type $\mathbb{T}$ of size s_1
- M_1 is stored as a linear array at address m_1 such that
 1. Address of $M_1[j] = m_1 + s_1 \times j$

Definition 3 A 2D matrix, M_2 , is defined using a type t_2 and positive integers n_X, n_Y , representing the dimensions/shape.

- $M_2[i]$ is a 1D matrix of length n_Y and type t_2
- $M_2[i, j]$ is a scalar of type t_2 and size s_2

- M_2 is stored as a linear array at address m_2 such that

1. Address of $M_2[i] = m_2 + s_2 \times (i \times n_Y)$
2. Address of $M_2[i, j] = m_2 + s_2 \times (i \times n_Y + j)$

Definition 4 A 3D matrix, $\mathbf{M}$, is defined using a type $\mathbf{T}$ and positive integers n_X, n_Y, n_Z , representing the dimensions/shape.

- $\mathbf{M}[x]$ is a 2D matrix of dimensions n_X, n_Y and type $\mathbf{T}$
- $\mathbf{M}[x, y]$ is a vector of length n_Z and type $\mathbf{T}$
- $\mathbf{M}[x, y, z]$ is a scalar of type $\mathbf{T}$
- $\mathbf{M}$ is stored as a linear array at address $m_{\mathbf{M}}$ such that
 1. Address of $\mathbf{M}[x] = m_{\mathbf{M}} + x \times (n_Y \times n_Z)$
 2. Address of $\mathbf{M}[x, y] = m_{\mathbf{M}} + (x \times n_Y \times n_Z) + (y \times n_Z)$
 3. Address of $\mathbf{M}[x, y, z] = m_{\mathbf{M}} + (x \times n_X \times n_Y) + (y \times n_Z) + z$

2 State

The state is represented by the data structures listed in Table 2

Name	Shape	Comment	n_X	n_Y	n_Z
x	1D array	activation at current timestamp	n_D		
xb	1D array	like x but inside residual branch	n_D		
xb2	1D array	like xb	n_D		
sq	1D array	n_H			
sk	1D array	n_D			
sk	1D array	n_D			
KC	3D array	keys of kv-cache	n_L	n_S	n_S
KV	3D array	values of kv-cache	n_L	n_S	n_S

Table 2: Data structures for state

3 Weights

The weights are represented by the data structures listed in Table 3

4 Code

Given

1. an integer p where $0 \leq p < n_P$
2. an integer t where $0 \leq t < n_V$

4.1 rmsnorm

Name	Shape	Comment	n_X	n_Y	n_Z
tet	2D array	Lookup table maps token IDs to vectors	n_V	n_D	
wk	3D array	key projections	n_L	n_D	n_{H_1}
wv	3D array	value projections	n_L	n_D	n_{H_1}

Table 3: Data structures for weights

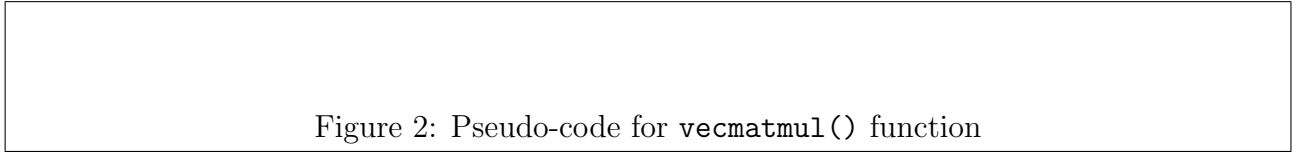
```

function rmsnorm(
     $x$ , /* type is  $(M_1, n, \text{float})$ 
     $w$  /* type is  $(M_1, n, \text{float})$ 
)
begin
     $y$  /* type is  $(M_1, n, \text{float})$ 
     $\epsilon = 10^{-5}$  /* type is  $(M_0, \text{float})$  */
     $s \leftarrow \frac{1}{\sqrt{\epsilon + \frac{\sum x_i^2}{n}}}$ 
     $y_i \leftarrow w_i \times x_i \times s$ 
    return  $y$ 
end

```

Figure 1: Pseudo-code for `rmsnorm()` function

4.2 vecmatmul



4.3 forward

```
function foo (  
    integer t,  
    integer p  
)  
begin  
    foreach  $l \in \{0, 1, \dots, n_L - 1\}$  do  
         $x \leftarrow \text{tet}[t]$   
         $\text{xb} \leftarrow \text{rmsnorm}(x, \text{wrms}[l])$   
         $\text{q}[l] \leftarrow \text{vecmatmul}(\text{xb}, \text{wq}[l])$   
         $\text{k}[l] \leftarrow \text{vecmatmul}(\text{xb}, \text{wk}[l])$   
         $\text{v}[l] \leftarrow \text{vecmatmul}(\text{xb}, \text{wv}[l])$   
  
    endfor  
end
```

Figure 3: Pseudo-code for `forward()` function

Function	Section Implemented
rmsnorm	Section 1

Table 4: Listing of functions

5 Instructions to LLM

For each of the functions listed in Table 4

1. Write an optimized C function using AVX2 intrinsics and SIMD instructions whenever possible.
2. Return a `.c` file — the definition
3. Return a `.h` file — the declaration